CHULA ENGINEERING
Foundation toward Innovation
COMPUTER

Data Mining Group
Machine Intelligence and Knowledge Discovery Lab
Chulalongkorn University

RNN

Prof. Peerapon Vateekul, Ph.D.
Department of Computer Engineering,
Faculty of Engineering, Chulalongkorn University
Peerapon.v@chula.ac.th
www.cp.eng.chula.ac.th/~peerapon/

Recurrent Neural Networks (RNN)

Note: Today's lecture introduces several DL models so that you can understand how to select the right one for your problem. You'll learn more in depth in the AI course next semester.

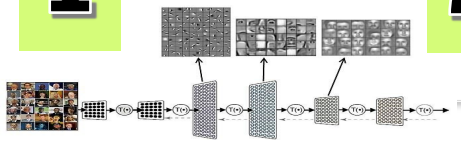
Deep Learning Application

Speech Recognition

Computer Vision

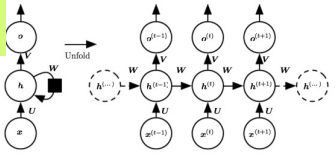
Natural Language Processing

1



CNN

2



RNN (LSTM)

Time Series Forecasting

Top 10 Strategic Technology Trends for 2020

People-centric

- Hyperautomation
- Multiexperience
- Democratization
- Human Augmentation
- Transparency and Traceability

Smart spaces

- Empowered Edge
- Distributed Cloud
- Autonomous Things
- Practical Blockchain
- AI Security

gartner.com/SmarterWithGartner

Source: Gartner
© 2019 Gartner, Inc. and/or its affiliates. All rights reserved. CTMKT_744630

Gartner

Gartner Top Strategic Technology Trends for 2021

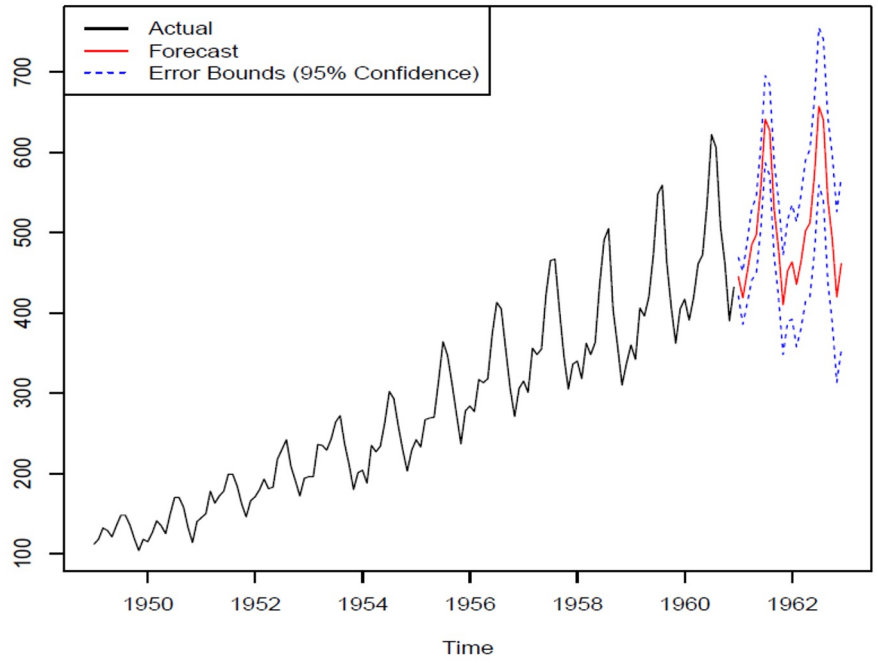
People centricity	Location independence	Resilient delivery
Internet of Behaviors	Distributed cloud	Intelligent composable business
Total experience strategy	Anywhere operations	AI engineering
Privacy-enhancing computing	Cybersecurity mesh	Hyperautomation

Combinatorial innovation

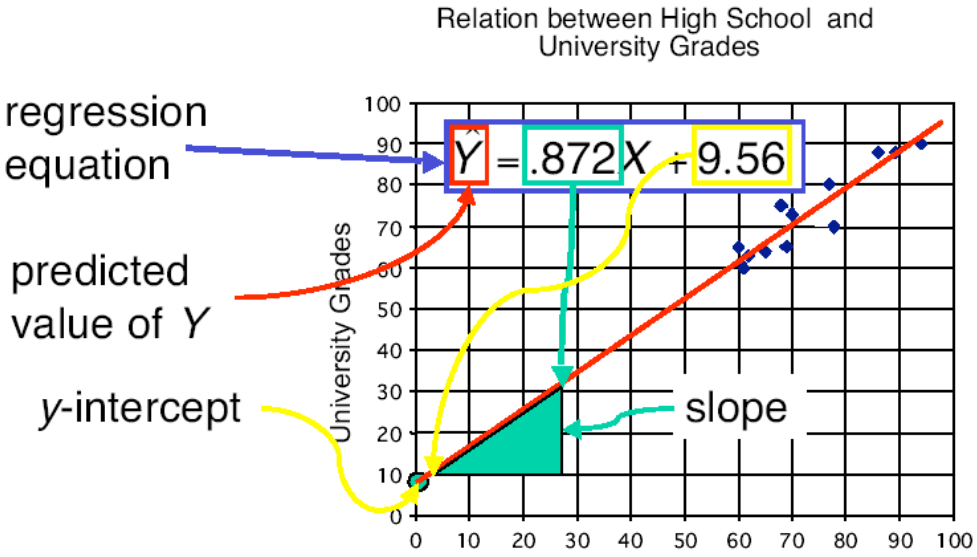
gartner.com/SmarterWithGartner

Source: Gartner
© 2020 Gartner, Inc. and/or its affiliates. All rights reserved. CTMKT_1026461

Gartner



1) Regression – Linear Relationship



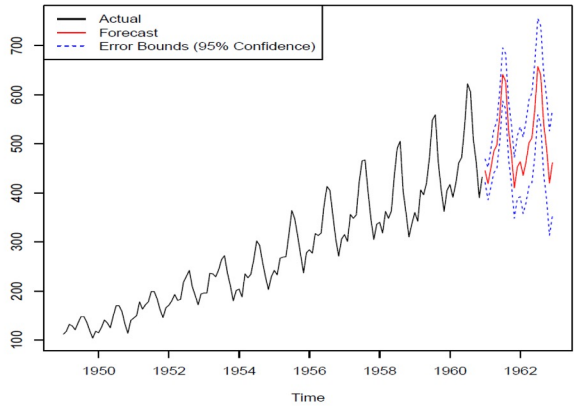
weight, coefficient

$$\underset{\text{target}}{\hat{y}} = \underset{\text{intercept}}{\hat{w}_0} + \hat{w}_1 x_1 + \hat{w}_2 x_2 \quad \text{(input)}$$

- **The least square method** aims to minimize the following term

$$\sum_{\substack{training \\ data}} (y_i - \hat{y}_i)^2$$

2) Autoregressive Model – Linear Relationship



- Based on linear regression, but using previous timestep data

$$X_t = c + \Sigma_{i=1}^{p} \varphi_i X_{t-i}$$

- Where X_t is data at timestep t, c is constant, and $\varphi_1, \varphi_2, \varphi_3, \ldots, \varphi_p$ are parameters for each timestep data

Example: When p = 2 (looking back two steps), the equation will be

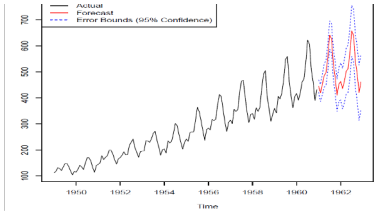
$$X_t = c + \varphi_1 X_{t-1} + \varphi_2 X_{t-2}$$

- Parameters are able to calculate using various method, such as ordinary least square procedure or Yule–Walker equations

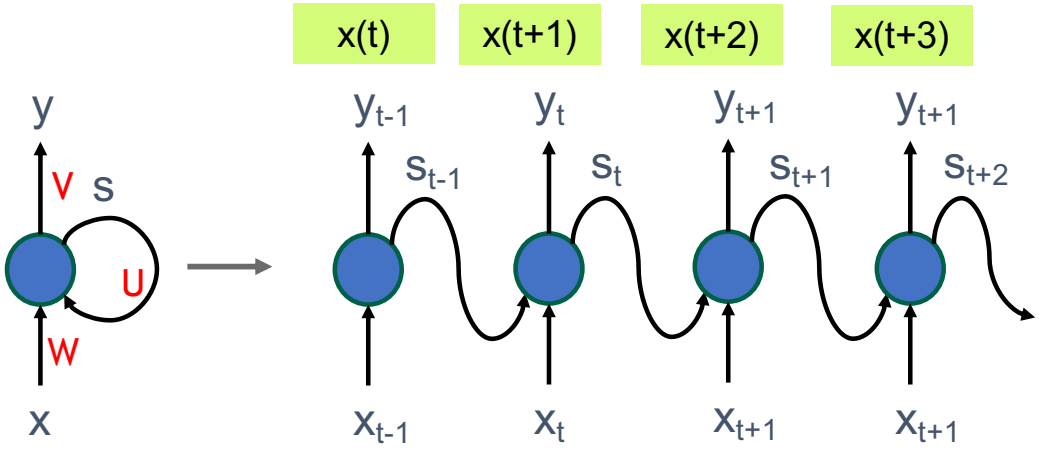
$$x(t+1) = \hat{w}_0 + \hat{w}_1 x(t) + \hat{w}_2 x(t-1)$$

3) Recurrent Neural Networks (RNN)

Non-Linear Relationship

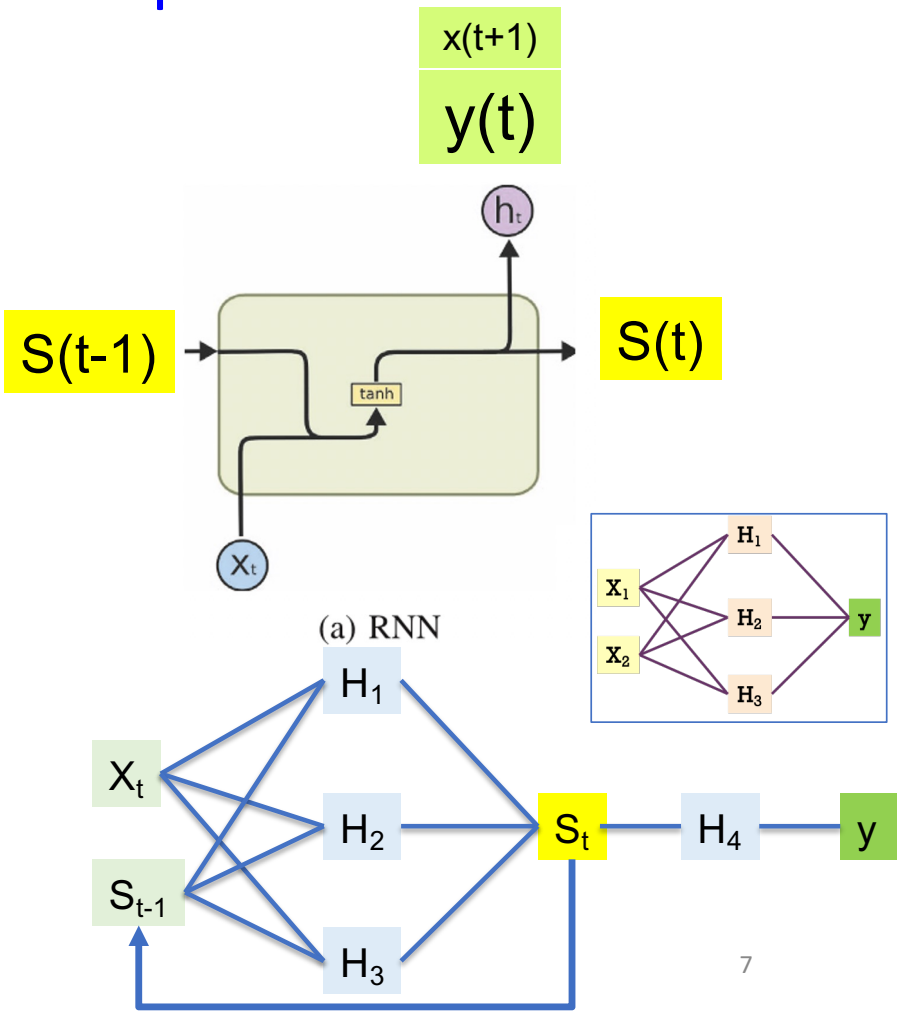


Forward Pass



$$s_t = \tanh\left(Ux_t + Ws_{t-1}\right)$$

x(t+1) $$\hat{y}_t = softmax\left(Vs_t\right)$$



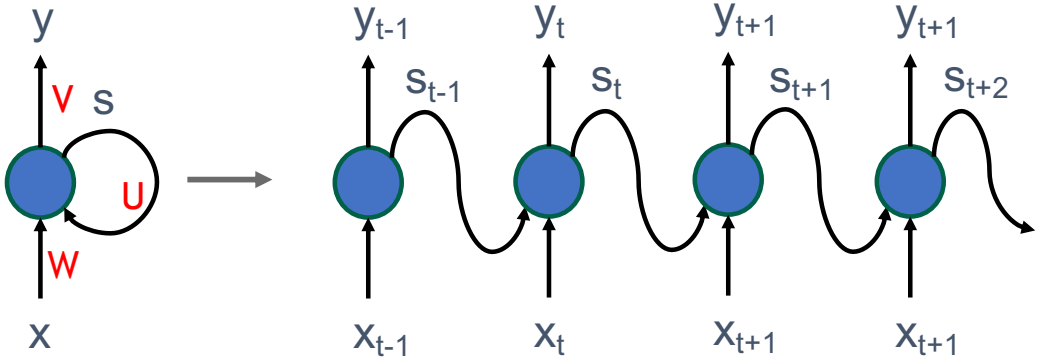
Reference: https://www.cse.iitk.ac.in/users/sigml/lec/Slides/LSTM.pdf

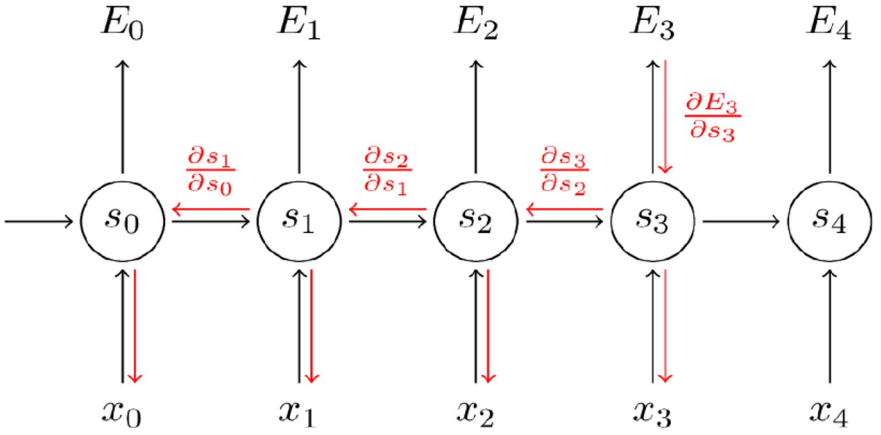
3) Recurrent Neural Networks (RNN)

Ggradient vanishing
= multiplication

Forward Pass

Backward Pass



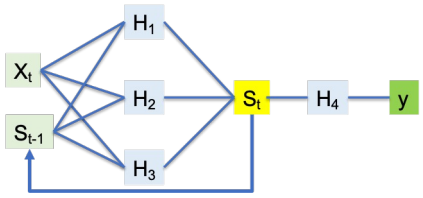


Unrolling in Time

Backpropagation-Through-Time (BPTT)

$$s_t = \tanh\left(Ux_t + Ws_{t-1}\right)$$

$$\hat{y}_t = softmax\left(Vs_t\right)$$

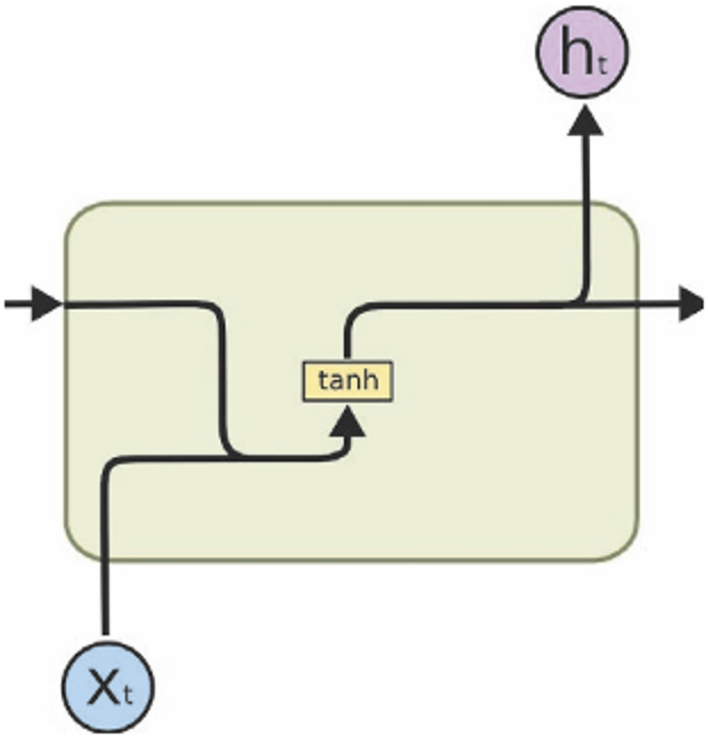


$$\frac{\partial E}{\partial W} = \sum_t \frac{\partial E_t}{\partial W}$$

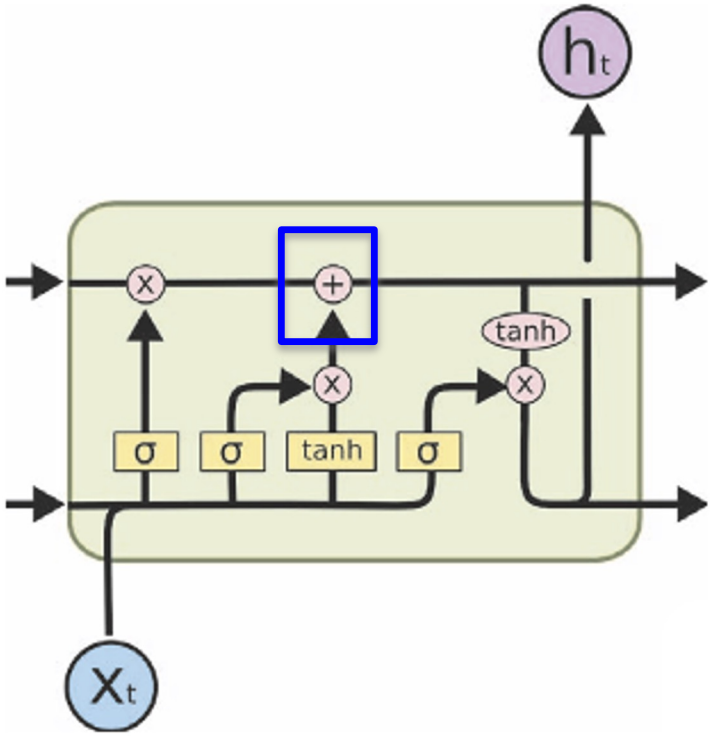
$$\frac{\partial E_3}{\partial W} = \sum_{k=0}^{3} \frac{\partial E_3}{\partial \hat{y}_3} \frac{\partial \hat{y}_3}{\partial s_3} \frac{\partial s_3}{\partial s_k} \frac{\partial s_k}{\partial W}$$

Reference: https://www.cse.iitk.ac.in/users/sigml/lec/Slides/LSTM.pdf

4) Long-Short Term Memory (LSTM)



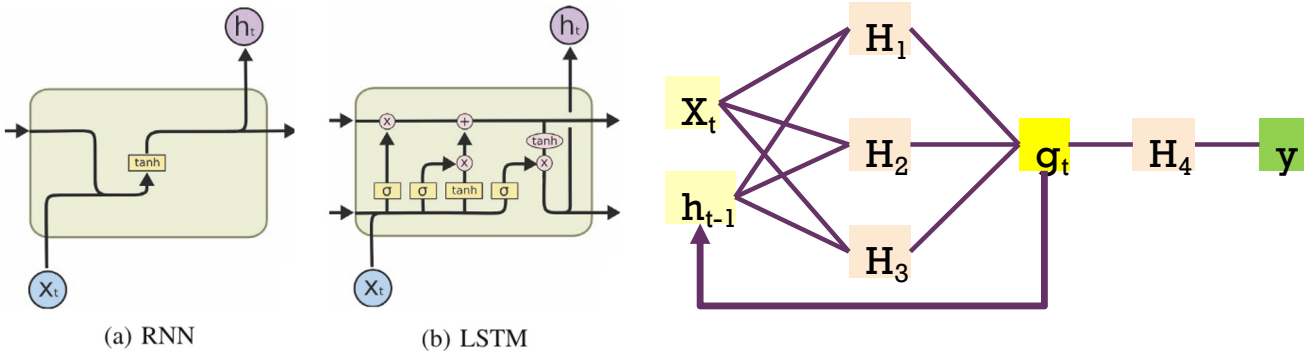
(a) RNN



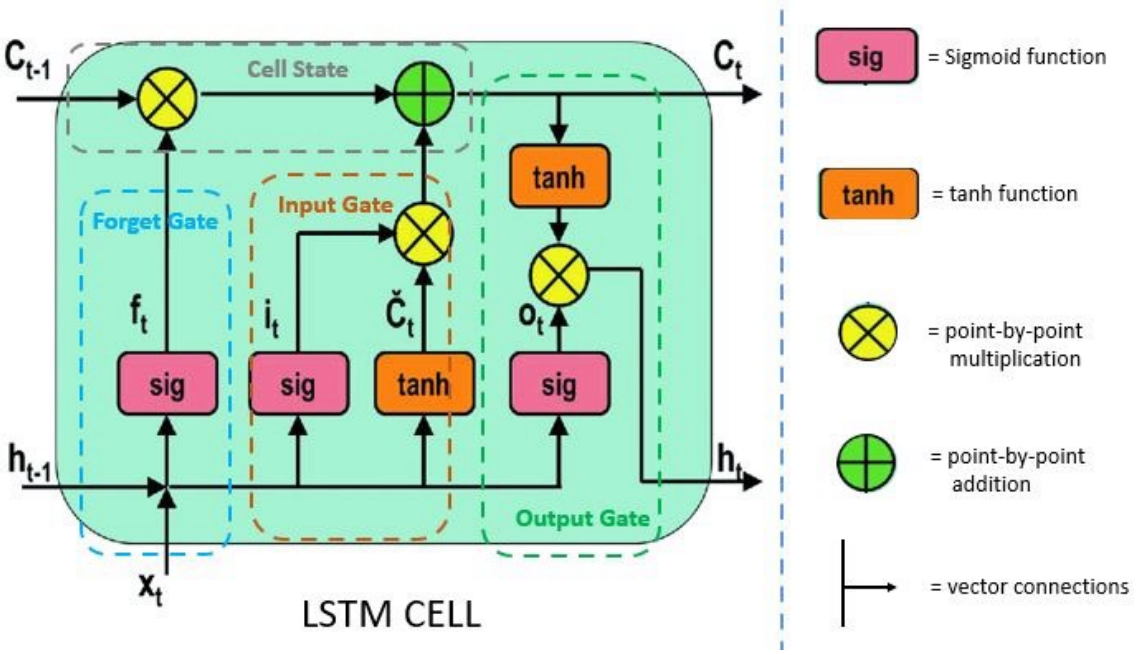
(b) LSTM

Reference: https://cs.uwaterloo.ca/~mli/Deep-Learning-2017-Lecture6RNN.ppt

Inside LSTM



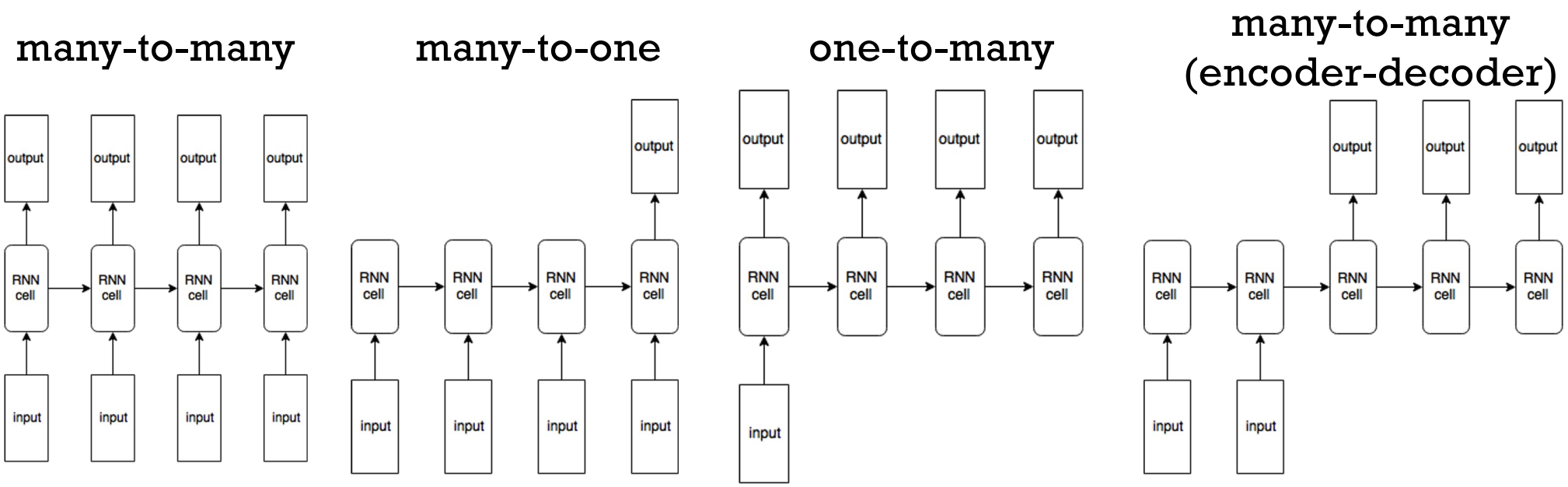
(a) RNN (b) LSTM



There are 3 gates with 2 outputs.

$$\mathbf{i}_{(t)} = \sigma(\mathbf{W}_{xi}^{T} \cdot \mathbf{x}_{(t)} + \mathbf{W}_{hi}^{T} \cdot \mathbf{h}_{(t-1)} + \mathbf{b}_i)$$
$$\mathbf{f}_{(t)} = \sigma(\mathbf{W}_{xf}^{T} \cdot \mathbf{x}_{(t)} + \mathbf{W}_{hf}^{T} \cdot \mathbf{h}_{(t-1)} + \mathbf{b}_f)$$
$$\mathbf{o}_{(t)} = \sigma(\mathbf{W}_{xo}^{T} \cdot \mathbf{x}_{(t)} + \mathbf{W}_{ho}^{T} \cdot \mathbf{h}_{(t-1)} + \mathbf{b}_o)$$
$$\mathbf{g}_{(t)} = \tanh(\mathbf{W}_{xg}^{T} \cdot \mathbf{x}_{(t)} + \mathbf{W}_{hg}^{T} \cdot \mathbf{h}_{(t-1)} + \mathbf{b}_g)$$
$$\mathbf{c}_{(t)} = \mathbf{f}_{(t)} \otimes \mathbf{c}_{(t-1)} + \mathbf{i}_{(t)} \otimes \mathbf{g}_{(t)}$$
$$\mathbf{y}_{(t)} = \mathbf{h}_{(t)} = \mathbf{o}_{(t)} \otimes \tanh(\mathbf{c}_{(t)})$$

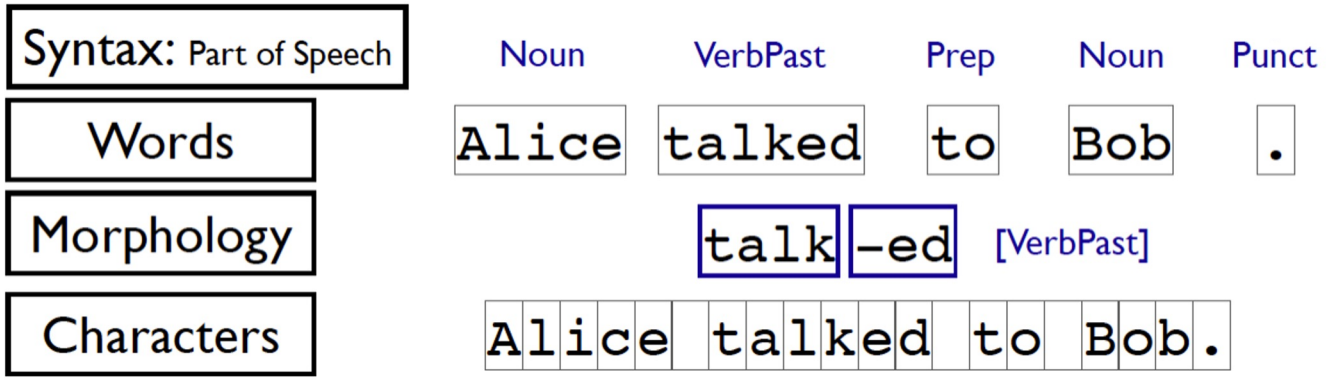
RNN in NLP Application

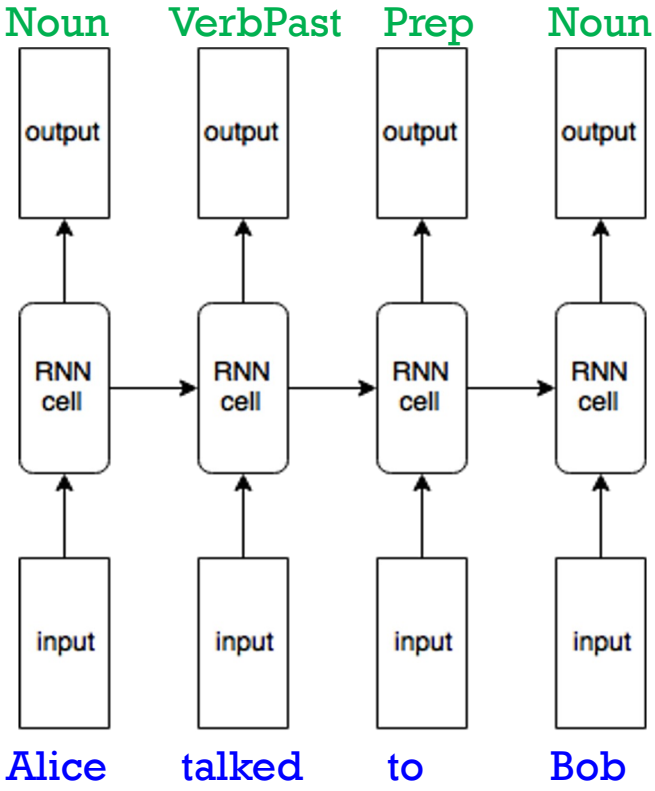


Reference: The Unreasonable Effectiveness of Recurrent Neural Networks, http://karpathy.github.io/2015/05/21/rnn-effectiveness/

1) Many-to-Many (e.g., PoS Tagging)

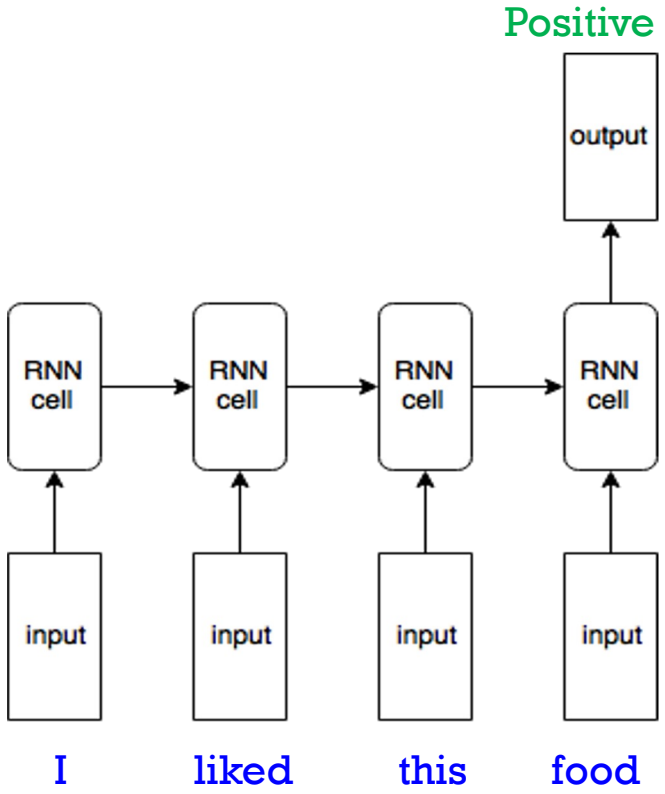
- You have seen and implemented this type of RNN architecture in your homework already.
- E.g., Tokenization, POS tagging
- Sequence Input, Sequence Output





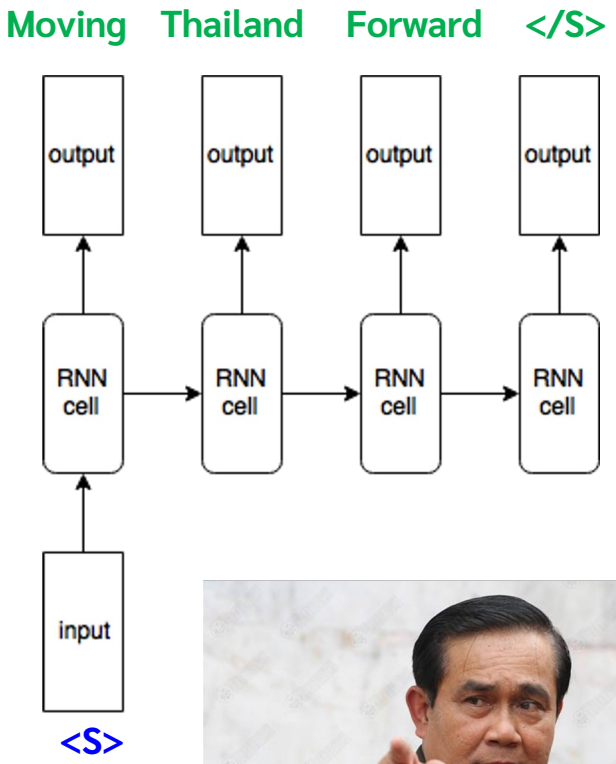
2) Many-to-One (e.g., Text Classification)

- E.g., sentiment analysis, text classification
- Sequence input, scalar output



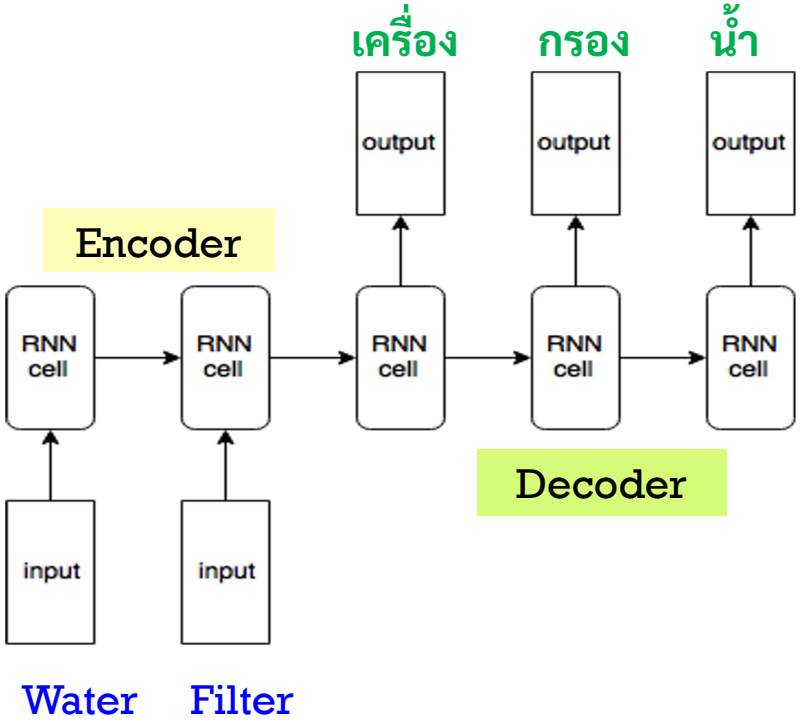
3) One-to-Many (Text Generation)

- Scalar input & Sequence output
- E.g., Music Generation, Image caption generation
- Music **generation**
  - Input: Initial seed
  - Output: Sequence of music notes
- Image caption **generation**
  - Input: Image features extracted by CNN
  - Output: Sequence of text

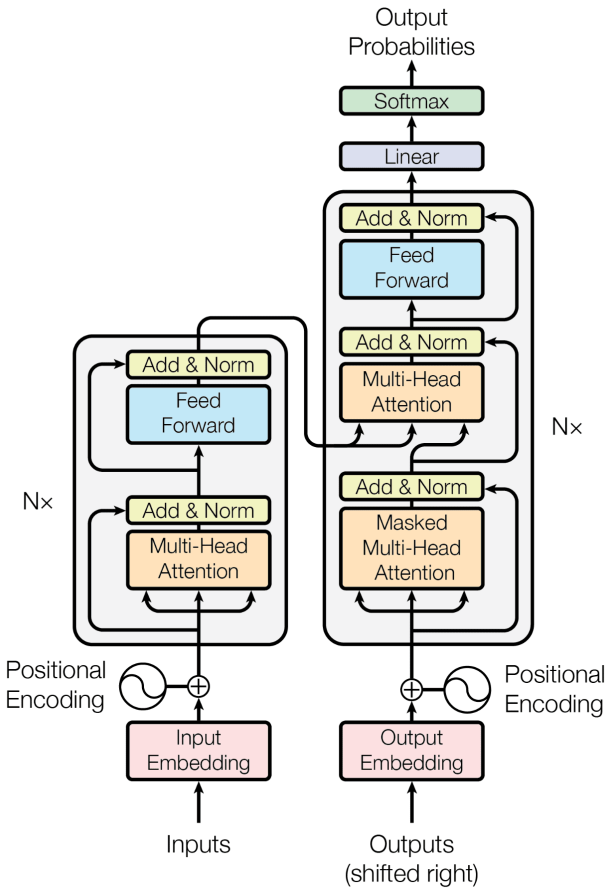


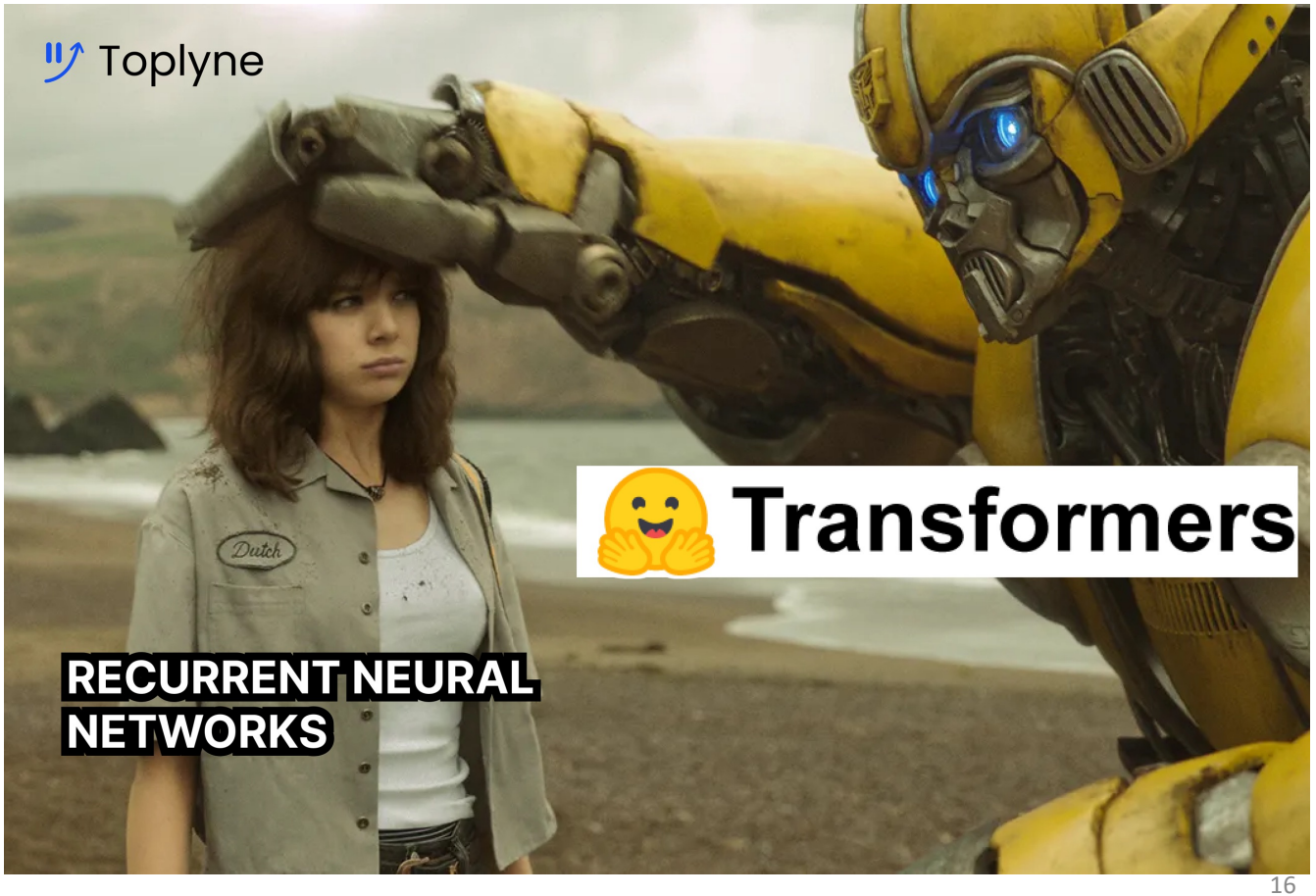
4) Many-to-Many (encoder-decoder) (e.g., Machine Translation)

- Sequence Input, Sequence output
- These two sequences can be of different length
- E.g., Machine Translation
  - Input: English Sentence
  - Output: Thai Sentence
- Machine Translation is also a **text generation task**



Rise of Transformer (2017)





+ Thank you
& any questions